Formation Dev / DevOps ▪ Outillage

Annexes techniques

Git · Docker · Kubernetes · Helm

Présentation · installation (Windows / macOS / Linux) · fonctionnalités · commandes

Bassem Ben Hamed ▪ 10 juillet 2026

Plan des annexes

1. **Git** — gestion de versions
   - Présentation & modèle des 3 zones
   - Installation, fonctionnalités, commandes
2. **Docker** — conteneurisation
   - Conteneurs vs VM, architecture
   - Installation, images, Dockerfile, Compose
3. **Kubernetes** — orchestration
   - Architecture cluster & objets clés
   - kubectl : installation et commandes
4. **Helm** — packaging K8s
   - Charts, releases, values
   - Installation et cycle de vie d'une release



Ces quatre outils forment la chaîne DevOps de référence : **versionner** (Git) → **empaqueter** (Docker) → **orchestrer** (Kubernetes) → **déployer** (Helm).

Annexes techniques

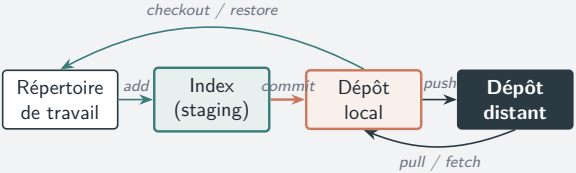
Git



Git — qu'est-ce que c'est ?

- **Gestionnaire de versions distribué** (DVCS), créé en 2005 par Linus Torvalds pour le noyau Linux.
- Chaque développeur possède une **copie complète** de l'historique — travail hors-ligne, pas de serveur obligatoire.
- Intégrité garantie : chaque commit est identifié par un **hash SHA** de son contenu.
- Standard de facto : GitHub, GitLab, Bitbucket, Azure DevOps s'appuient tous sur Git.

Le modèle des 3 zones (+ le dépôt distant)



Une modification traverse ces zones : on choisit ce qui entre dans le commit (staging), puis on synchronise avec le distant.

Git — installation (Windows / macOS / Linux)

Windows — Git for Windows (inclut Git Bash) :

```
winget install --id Git.Git -e
# ou : installeur graphique depuis https://git-scm.com/download/win
```

macOS — via Homebrew (ou les outils Xcode) :

```
brew install git          # ou : xcode-select --install
```

Linux — via le gestionnaire de paquets :

```
sudo apt install git      # Debian / Ubuntu
sudo dnf install git      # Fedora / RHEL
```

Configuration initiale (obligatoire, une seule fois) :

```
git config --global user.name  "Prénom Nom"
git config --global user.email "prenom.nom@oddo-bhf.com"
git config --global init.defaultBranch main
git --version             # vérifier l'installation
```

Git — fonctionnalités clés

Historique & intégrité

- **Commits** : instantanés atomiques, horodatés, signables (GPG).
- **Tags** : jalons immuables pour les versions (`v1.2.0`).
- Historique complet en local → `log`, `diff`, `blame` instantanés.

Collaboration

- **Remotes** multiples (origin, fork…).
- Pull requests / merge requests (côté forge).
- **Hooks** : scripts déclenchés sur commit, push…

Branches : la force de Git

- Création **instantanée et légère** (un simple pointeur).
- Fusion par **merge** (conserve l'historique) ou **rebase** (historique linéaire).
- Base des workflows d'équipe : GitFlow, trunk-based, feature branches.

Filet de sécurité

- **stash** : mettre de côté un travail en cours.
- **revert** : annuler proprement un commit publié.
- **reflog** : retrouver presque tout ce qui semble perdu.

Git — commandes : démarrer et travailler au quotidien

Créer ou récupérer un dépôt

```
git init                                # nouveau dépôt dans le dossier courant
git clone https://github.com/org/projet.git
```

Le cycle quotidien

```
git status                              # état des fichiers (modifiés, indexés...)
git add Program.cs                      # indexer un fichier (ou : git add .)
git commit -m "feat: ajout du endpoint /orders"
git push                                # publier vers le dépôt distant
git pull                                # récupérer + fusionner les nouveautés
git fetch                               # récupérer sans fusionner (inspection)
```

Convention : messages de commit courts et normés (`feat:`, `fix:`, `docs:`, `refactor:`...) — l'historique devient lisible et exploitable (changelogs, revues).

Git — commandes : branches, fusion, tags

```
git branch                          # lister les branches locales
git switch -c feature/paiement      # créer et basculer sur une branche
git switch main                     # revenir sur main
git merge feature/paiement          # fusionner la branche dans main
git rebase main                     # rejouer ses commits sur main (historique linéaire)
git branch -d feature/paiement      # supprimer la branche fusionnée
git tag -a v1.2.0 -m "release Q3"   # poser un tag annoté
git push --tags                     # publier les tags
```

merge

Crée un commit de fusion ; l'historique reflète la réalité du travail en parallèle. Sûr par défaut.

rebase

Historique linéaire et propre, mais **réécrit les commits** : jamais sur une branche déjà partagée.

Git — commandes : inspecter et réparer

Inspecter

```
git log --oneline --graph --all   # historique compact avec les branches
git diff                          # modifs non indexées (--staged : indexées)
git show a1b2c3d                  # détail d'un commit
git blame Program.cs              # qui a modifié quelle ligne
```

Réparer sans casser

```
git stash                         # mettre de côté le travail en cours
git stash pop                     # le restaurer
git restore Program.cs            # annuler les modifs locales d'un fichier
git revert a1b2c3d                # commit inverse (sûr sur une branche partagée)
git reset --hard HEAD~1           # supprimer le dernier commit (local uniquement
    !)
git reflog                        # journal de secours : retrouver un état perdu
```

Règle d'or : `revert` pour ce qui est déjà poussé, `reset` seulement pour ce qui est resté local.

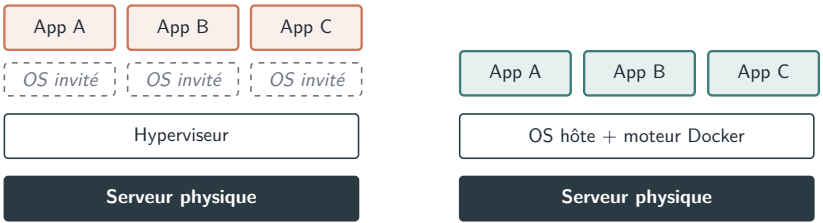
Annexes techniques

Docker



Docker — qu'est-ce que c'est ?

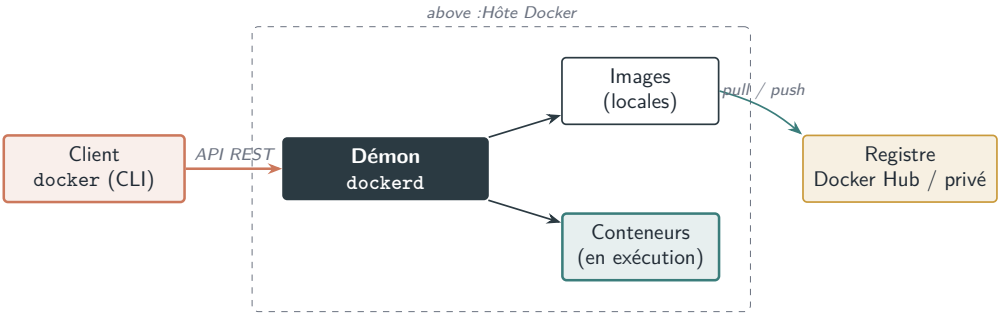
- Plateforme de **conteneurisation** (2013) : empaqueter une application **avec toutes ses dépendances** dans une image portable, exécutée de façon isolée.
- Résout le « ça marche sur ma machine » : la même image tourne à l'identique du poste dev à la production.



Machines virtuelles *: un OS complet par app (lourd, minutes)* ***Conteneurs*** *: noyau partagé, isolation par namespaces (léger, secondes)*

Un conteneur démarre en **quelques secondes**, pèse quelques dizaines de Mo, et l'on en fait tourner des dizaines là où une VM en supporterait quelques-unes.

Docker — architecture : client, démon, registre



Image

Modèle **immuable** en couches, décrit par un `Dockerfile`. Versionnée par tag (`app:1.4`).

Conteneur

Instance **vivante** d'une image : processus isolé (namespaces, cgroups), éphémère par défaut.

Registre

Bibliothèque d'images partagées : Docker Hub, GitHub Container Registry, registre privé d'entreprise.

Docker — installation (Windows / macOS / Linux)

Windows — Docker Desktop (nécessite WSL 2) :

```
wsl --install                          # une fois, puis redémarrer
winget install Docker.DockerDesktop
```

macOS — Docker Desktop (Intel ou Apple Silicon) :

```
brew install --cask docker             # puis lancer Docker.app
```

Linux — Docker Engine natif (sans Desktop) :

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER          # utiliser docker sans sudo (se reconnecter)
```

Vérification :

```
docker --version
docker run hello-world                 # télécharge et exécute une image de test
```

En entreprise : Docker Desktop est soumis à licence ; alternatives possibles côté poste de travail (Podman, Rancher Desktop, Colima).

Docker — fonctionnalités clés

Images en couches

Chaque instruction du `Dockerfile` crée une couche **mise en cache** et réutilisée → builds rapides, images mutualisées. Le **multi-stage build** sépare compilation et exécution pour des images finales minimales.

Volumes & persistance

Le conteneur est éphémère ; les données durables vivent dans des **volumes** (`-v donnees:/var/lib/...`) ou des montages du poste (`-v ./src:/app`).

Réseaux

Réseaux virtuels isolés ; les conteneurs d'un même réseau se joignent **par leur nom** (DNS intégré). Publication de ports vers l'hôte : `-p 8080:80`.

Docker Compose

Décrire une **pile multi-conteneurs** (API + base + cache) dans un seul `compose.yaml` et la piloter d'une commande — l'environnement de dev reproductible de toute l'équipe.

Docker — commandes : images

```
docker pull nginx:1.27              # télécharger une image depuis le registre
docker images                       # lister les images locales
docker build -t monapp:1.0 .        # construire depuis le Dockerfile courant
docker tag monapp:1.0 registry.interne/monapp:1.0
docker push registry.interne/monapp:1.0
docker history monapp:1.0           # inspecter les couches
docker rmi monapp:1.0               # supprimer une image locale
docker image prune -a               # nettoyer les images inutilisées
```

Tag = `registre/depot:version`. Sans registre : Docker Hub implicite ; sans version : `:latest` (à éviter en production — préférer des versions explicites et reproductibles).

Docker — commandes : conteneurs

```
docker run -d --name web -p 8080:80 nginx:1.27
#          -d : détaché   --name : nom   -p hôte:conteneur : port publié
docker ps                              # conteneurs actifs (-a : tous)
docker logs -f web                     # suivre les logs
docker exec -it web bash               # ouvrir un shell dans le conteneur
docker stats                           # CPU / mémoire en direct
docker stop web && docker rm web       # arrêter puis supprimer
```

Options de `run` à connaître

- `-e CLE=valeur` : variable d'environnement (config 12-factor).
- `-v volume:/chemin` : persistance des données.
- `-network mon-reseau` : rattacher à un réseau nommé.
- `-rm` : suppression automatique à l'arrêt (tests jetables).

Docker — Dockerfile : exemple .NET multi-stage

```
# --- étape 1 : compilation (SDK complet) ---
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish Catalog.Api -c Release -o /app

# --- étape 2 : exécution (runtime minimal) ---
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app .
EXPOSE 8080
ENTRYPOINT ["dotnet", "Catalog.Api.dll"]
```

```
docker build -t catalog-api:1.0 .
docker run -d -p 8080:8080 catalog-api:1.0
```

Le multi-stage garde le SDK ($\approx$ 800 Mo) hors de l'image finale : seul le runtime ($\approx$ 220 Mo) part en production.

Docker — Compose : une pile complète en un fichier

`compose.yaml` :

```yaml
services:
  api:
    build: ./src/Catalog.Api
    ports: ["8080:8080"]
    environment:
      - ConnectionStrings__db=Host=db
    depends_on: [db]
  db:
    image: postgres:17
    environment:
      - POSTGRES_PASSWORD=devpass
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Commandes

```
docker compose up -d
docker compose ps
docker compose logs -f api
docker compose down
# -v : supprime aussi les volumes
docker compose down -v
```

Cas d'usage

Environnement de dev complet (API + base + cache) démarré en une commande, identique pour toute l'équipe — le pendant local de ce que Kubernetes fera en production.

Annexes techniques

Kubernetes



Kubernetes — qu'est-ce que c'est ?

- **Orchestrateur de conteneurs** open source (Google, 2014 ; aujourd'hui CNCF) — abrégé **K8s**.
- Gère le cycle de vie des conteneurs **à l'échelle d'un cluster** de machines : placement, redémarrage, montée en charge, réseau, configuration.
- Modèle **déclaratif** : on décrit l'état souhaité en YAML, Kubernetes **réconcilie** en permanence l'état réel vers cet état cible.

Auto-réparation

Un conteneur tombe ? Il est relancé. Un nœud meurt ? Les pods sont replanifiés ailleurs. Sondes de santé (liveness / readiness) intégrées.

Montée en charge

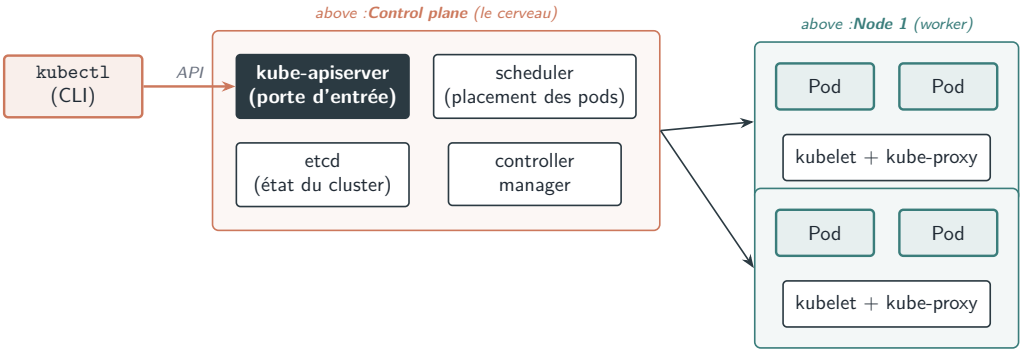
Réplication horizontale manuelle (`scale`) ou automatique (HPA, selon CPU / métriques). Répartition de charge native entre les répliques.

Déploiements maîtrisés

Rolling update sans coupure, rollback en une commande, stratégies blue/green et canary possibles.

Contexte bancaire : la réconciliation déclarative + les manifestes versionnés dans Git = auditabilité et reproductibilité des déploiements (GitOps).

Kubernetes — architecture d'un cluster



- **Control plane** : reçoit les demandes (API server), stocke l'état (etcd), décide du placement (scheduler) et réconcilie (controllers).
- **Workers** : le `kubelet` fait tourner les pods, `kube-proxy` gère le réseau des Services.

Kubernetes — les objets à connaître

Objet	Rôle
Pod	Plus petite unité déployable : un ou plusieurs conteneurs qui partagent réseau et stockage. Éphémère.
Deployment	Maintient N répliques d'un pod, gère les rolling updates et les rollbacks. L'objet du quotidien.
Service	Adresse réseau **stable** + répartition de charge devant des pods éphémères (ClusterIP, NodePort, LoadBalancer).
Ingress	Routage HTTP(S) depuis l'extérieur vers les Services (hôtes, chemins, TLS).
ConfigMap / Secret	Configuration et données sensibles injectées dans les pods (variables, fichiers) — le code reste identique entre environnements.
Namespace	Cloisonnement logique du cluster (équipes, environnements, quotas).

Kubernetes — installation de `kubectl` et cluster local

kubectl (le client) — Windows / macOS / Linux :

```
winget install Kubernetes.kubectl                    # Windows
brew install kubectl                                 # macOS
sudo apt install kubectl                             # Linux (dépôt Kubernetes)
kubectl version --client                             # vérification
```

Un cluster local pour s'entraîner :

```
# Option 1 : Docker Desktop -> Settings -> Kubernetes -> Enable (Win / macOS)
# Option 2 : minikube (multiplateforme)
brew install minikube        # ou winget install Kubernetes.minikube
minikube start
# Option 3 : kind (Kubernetes in Docker, léger, idéal CI)
brew install kind && kind create cluster
```

```
kubectl get nodes                                    # le cluster répond
```

En production : clusters managés (AKS, EKS, GKE) ou distributions on-premise (OpenShift, Rancher, k3s).

Kubernetes — un manifeste YAML type

`deploiement.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: catalog }
  template:
    metadata:
      labels: { app: catalog }
    spec:
      containers:
      - name: api
        image: registry/catalog-api:1.0
        ports: [{ containerPort: 8080 }]
```

`service.yaml` :

```yaml
apiVersion: v1
kind: Service
metadata:
  name: catalog-api
spec:
  selector: { app: catalog }
  ports:
  - port: 80
    targetPort: 8080
```

Lecture

Le Deployment maintient **3 répliques** de l'image ; le Service expose une adresse stable `catalog-api:80` vers les pods portant le label `app: catalog`.

Kubernetes — commandes `kubectl` : le quotidien

```
kubectl apply -f deploiement.yaml      # créer / mettre à jour (déclaratif)
kubectl get pods                       # lister (-o wide : + de détails)
kubectl get all -n mon-namespace       # tout un namespace
kubectl describe pod catalog-api-xxx   # détails + événements (1er réflexe debug)
kubectl delete -f deploiement.yaml     # supprimer les objets du manifeste
```

Cycle de vie d'un Deployment

```
kubectl scale deployment catalog-api --replicas=5
kubectl set image deployment/catalog-api api=registry.interne/catalog-api:1.1
kubectl rollout status deployment/catalog-api    # suivre le rolling update
kubectl rollout undo deployment/catalog-api      # rollback en une commande
```

En pratique on privilégie **apply -f** (manifestes versionnés dans Git) aux commandes impératives : l'état du cluster reste traçable et rejouable.

Kubernetes — commandes `kubectl` : observer et déboguer

```
kubectl logs -f catalog-api-xxx            # logs d'un pod (-f : suivre)
kubectl logs -f deployment/catalog-api     # logs via le Deployment
kubectl exec -it catalog-api-xxx -- sh     # shell dans le conteneur
kubectl port-forward svc/catalog-api 8080:80   # tester en local
kubectl get events --sort-by=.lastTimestamp    # événements récents du cluster
kubectl top pods                           # consommation CPU / mémoire
kubectl config get-contexts                # sur quel cluster suis-je ?
kubectl config use-context prod            # changer de cluster / contexte
```

Réflexes de diagnostic

- Pod en `CrashLoopBackOff` → `logs` (l'appli plante au démarrage ?).
- Pod en `Pending` → `describe` (pas de ressources ? volume manquant ?).
- Pod en `ImagePullBackOff` → image ou credentials du registre incorrects.

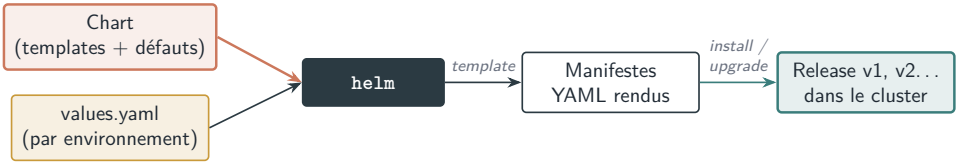
Annexes techniques

Helm



Helm — qu'est-ce que c'est ?

- Le **gestionnaire de paquets de Kubernetes** (projet CNCF) — l'équivalent d'`apt` / `brew` pour un cluster.
- Problème résolu : une appli K8s = des dizaines de manifestes YAML quasi identiques entre environnements. Helm les **regroupe et les paramètre**.
- Un **chart** = des templates YAML + des valeurs par défaut ; son installation dans un cluster = une **release**, versionnée et réversible.



Même chart, plusieurs `values` : dev, recette, production — seuls les paramètres changent (répliques, ressources, hôtes, secrets). Les charts se partagent via des **dépôts** (Artifact Hub, dépôt d'entreprise).

Helm — installation (Windows / macOS / Linux)

Windows :

```
winget install Helm.Helm            # ou : choco install kubernetes-helm
```

macOS :

```
brew install helm
```

Linux — script officiel ou paquet :

```
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
    bash
# ou : sudo snap install helm --classic
```

Vérification (Helm utilise le contexte `kubectl` courant) :

```
helm version
kubectl config current-context      # Helm ciblera ce cluster
```

Helm 3 n'a plus de composant serveur (Tiller) : un simple binaire côté client, qui parle à l'API Kubernetes avec vos droits.

Helm — anatomie d'un chart

```
monapp/
  Chart.yaml          # nom, version
  values.yaml         # défauts
  templates/
    deployment.yaml   # templatisé
    service.yaml
    ingress.yaml
    _helpers.tpl      # macros
  charts/             # dépendances
```

`helm create monapp` génère ce squelette complet, prêt à adapter.

Extrait de `templates/deployment.yaml` :

```
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image }}"
```

Et le `values.yaml` correspondant :

```
replicaCount: 3
image: registry.interne/catalog-api:1.0
```

Helm — commandes : cycle de vie d'une release

Dépôts et recherche

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo postgresql          # chercher un chart dans les dépôts
```

Installer, mettre à jour, revenir en arrière

```
helm install madb bitnami/postgresql -f values-dev.yaml
helm upgrade --install madb bitnami/postgresql -f values-dev.yaml
#            --install : crée si absent (idempotent, idéal CI/CD)
helm list                            # releases installées
helm history madb                    # versions successives de la release
helm rollback madb 1                 # revenir à la révision 1
helm uninstall madb
```

Valider avant de déployer

```
helm lint ./monapp                   # vérifier le chart
helm template ./monapp -f values-prod.yaml   # rendre le YAML sans l'appliquer
helm diff upgrade madb ./monapp      # plugin : diff avant upgrade
```

Récapitulatif — la chaîne complète



	Rôle	Unité de base	Commande signature
Git	versionner le code	commit	`git commit / push`
Docker	empaqueter l'application	image / conteneur	`docker build / run`
Kubernetes	orchestrer à l'échelle	pod / deployment	`kubectl apply -f`
Helm	packager le déploiement	chart / release	`helm upgrade -install`

✳ Avec Claude Code, ces quatre outils deviennent pilotables en langage naturel : générer un Dockerfile, expliquer un manifeste, déboguer un `CrashLoopBackOff` ou écrire un chart Helm complet.